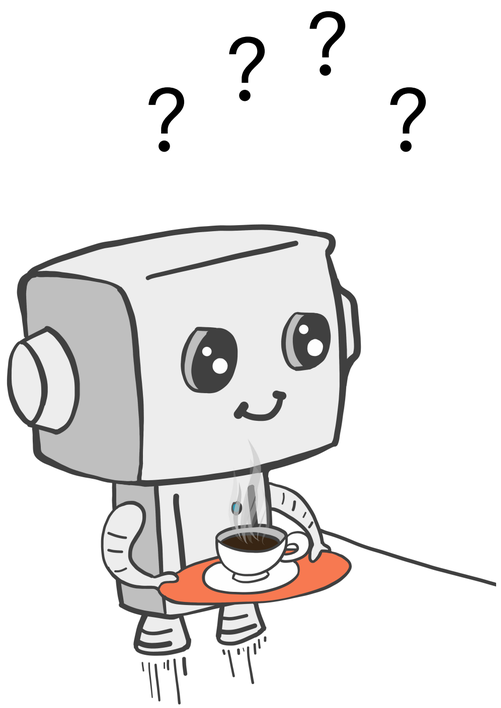
Introduction to PDDL

Darko Bozhinoski,

Ph.D. in Computer Science

Motivations

- What is the Planning Domain Definition Language (PDDL)?



Motivations

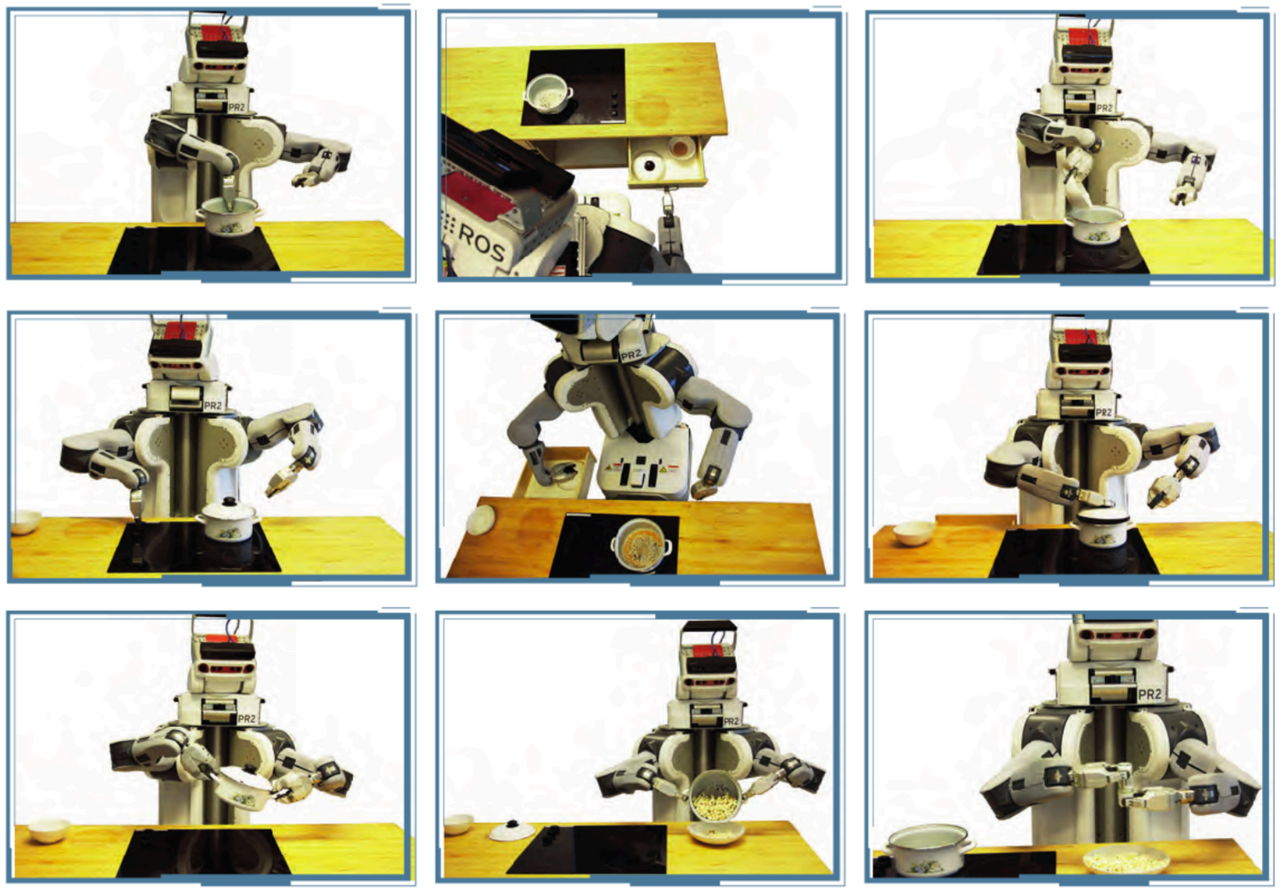


Fig. From [Beetz-2020]

- Formulated the problem:
  - First Order Logic (FOL)
- PROLOG

Motivations

- Schema for a Knowledge-enabled agent:
- **Action plan:**
  - Action sequencer: solves the planning problem
- **Execution of the plan:**
  - If the goal can be satisfied actions are executed

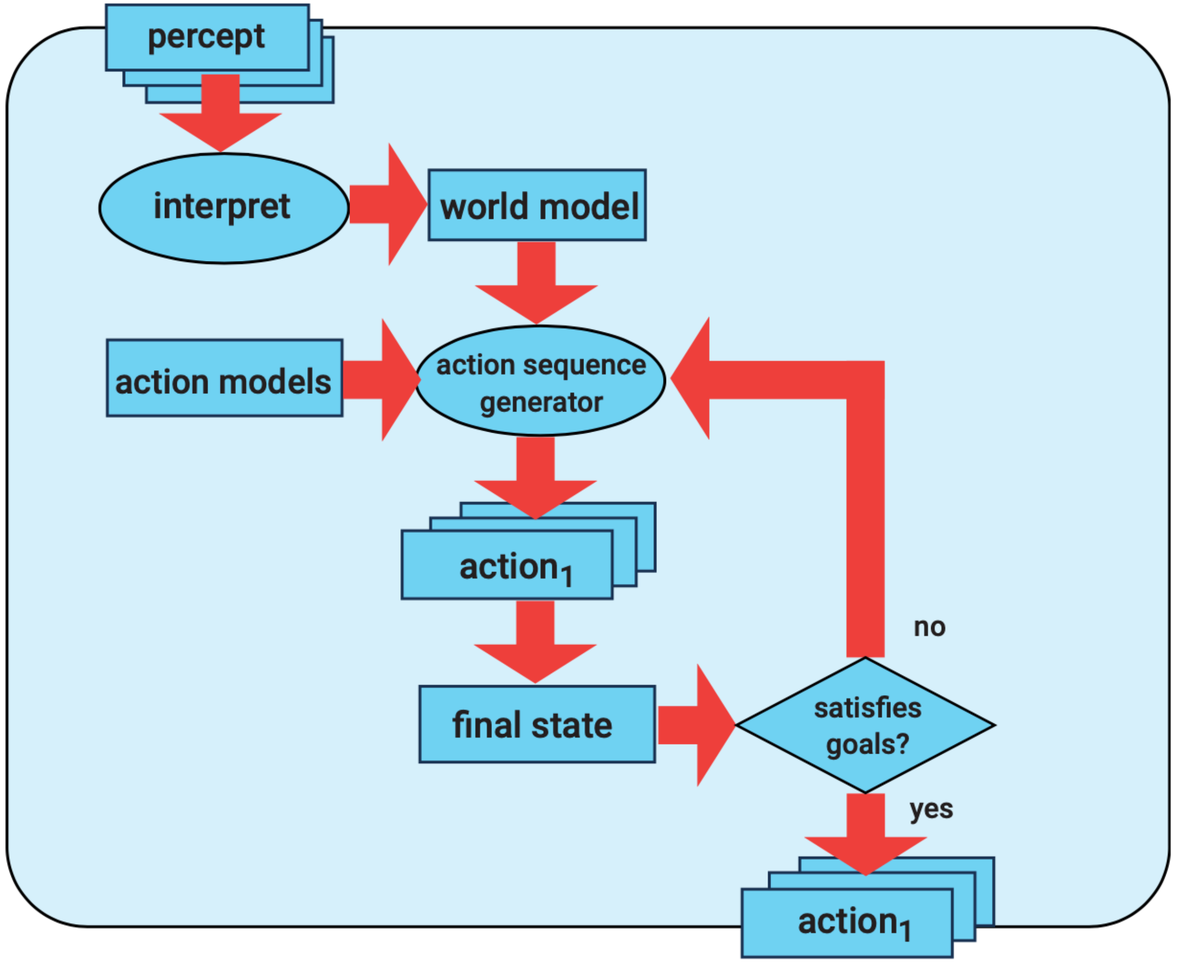


Fig. From [Beetz-2020]

Motivations

- PDDL:

  A framework for formulating and solving planning problem.

What is planning?

Intuitive definition:

Planning is the thinking before acting

More precise definition:

Planning is the process of choosing and scheduling actions that lead towards a goal, based on a high-level description of the world

Two families of planning approaches

Domain specific planning

- Methods and representations are specific to a problem:
  - Path and motion planning
  - Manipulation planning
  - ...

Domain independent planning

- Applicable across very different domains
- Many different flavours
  - Offline/online
  - Discrete/continuous
  - Deterministic/non-deterministic
  - Fully/partially observable
  - Sequential/temporal

Planning Domain Definition Language

We need to formally describe:

- The environment in which we operate: the initial state, and the goal
- The actions that our robot can do, and how these affect the environment



PDDL - The classical planning problem

The classical (simplest) PDDL planning problem is described by a tuple:

$$< F, A, I, G >$$

where:

- F is a set of boolean propositions (facts)
- A is a set of deterministic actions (with preconditions and effects)
- $I \in S$ is the initial state
- G is the goal state

The set of states in S is the power set of F, $S = 2^F$

PDDL - The classical planning problem

The classical PDDL assumes:

- Deterministic and instantaneous actions
- Boolean propositions to describe the world
- The world only changes according to the actions we apply (no external events)
- The system is fully observable (i.e. I is known)

PDDL - The classical planning problem

Classical PDDL might seem limiting:

- Assumptions might appear unrealistic for real use-cases

Why is it so popular then?

- It provides idealised settings to develop automated planners
- In many cases it is enough - simplified robotics task in controlled environments

Extensions:

- PDDL2.1, Probabilistic PDDL (PPDDL), RDDL, and many more...

PDDL - Structure

Planning tasks in classical PDDL relies on two separate files:

Domain file
domain.pddl

- **Predicates**: properties of the world
- **Operators**: the way in which the state can change
- First order logic (FOL): derive F and A

Problem file
problem.pddl

- **Objects**: the things in the world
- **Initial state**
- **Goal**

PDDL - Solvers

How to find a plan to achieve a goal?

- Solvers take as input a *domain.pddl* and *problem.pddl*, and generate a *plan*
- The planning problem can be seen as network of states, where actions represent transitions.
- It is a search problem: MCTS, A*...

Syntax and tools

Domain file
`domain.pddl`

PDDL - Syntax

```
(define (domain <domain-name>)
  (:requirements
       <requirements-list>)
  (:types
       <types-list>)
  (:predicates
       <predicate-list>)
  (:action
       <action-details>)
)
```

- `<domain-name>` is the name of the world
- `:requirements` - which features of PDDL are being used
- `:types` - optional, type of objects
- `:predicates` - list of model's state, true or false facts
- `:action` - list of possible state transitions

Problem file
`problem.pddl`

PDDL - Syntax

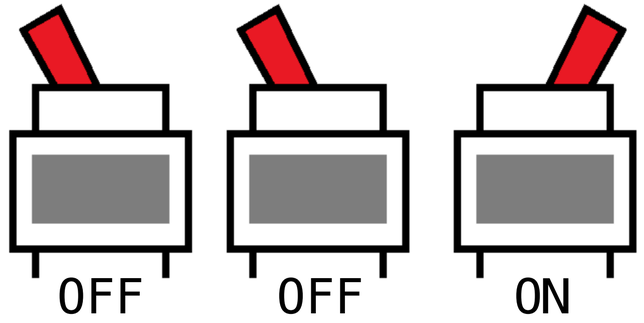
```
(define (problem <title>)
    (:domain <domain-name>)
    (:objects
        <object-list>)
    (:init
        <predicates>)
    (:goal
        <predicates>)
)
```

- `<title>` is the name you give to the problem
- `<domain-name>` refers to the relative domain.pddl
- `:objects` - things in the world you want to manipulate
- `:init` - the initial state of your objects
- `:goal` - final state(s) to be achieved

Classic PDDL - Example

domain.pddl

```
(define (domain simple_switches)
    (:requirements :typing)
    (:types switch)
    (:predicates (off ?s - switch)
                 (on ?s - switch))
    (:action switch_on
        :parameters (?s - switch)
        :precondition (off ?s)
        :effect (and (not (off ?s))
                     (on ?s))))
```



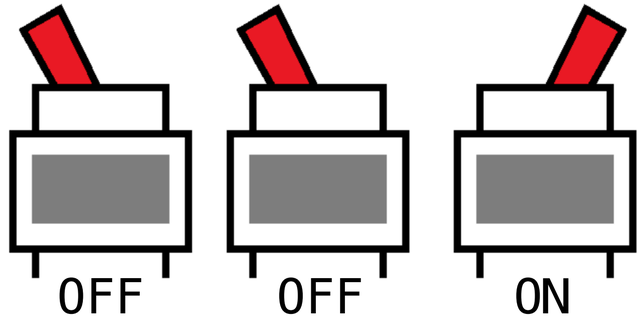
- An action has:
  - $par(a) \subseteq F$: parameters
  - $pre(a) \subseteq F$: simple conditions
  - $add(a) \subseteq F$: add effects
  - $del(a) \subseteq F$: delete effects
- The order of expressions is prefix
  - (operator <arguments>):

    (and <conjunct1> … <conjunctN>)

Classic PDDL - Example

problem.pddl

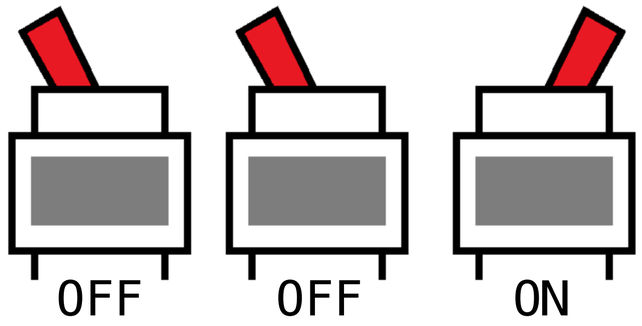
```
(define (problem switches_problem)
    (:domain simple_switches)
    (:objects s1 s2 s3 - switch)
    (:init (off s1) (off s2) (on s3))
    (:goal (and (on s1)
                (on s2)
                (on s3))) )
```



- The domain in which we operate is simple_switches
- There are 3 objects of type switch
- Two out of three switches are OFF
- The goal is to have all the switches ON
- Different problems, same domain!!!

Classic PDDL - Example

A plan for a classical planning problem is a sequence of actions that are applicable from the initial state and lead to a a state that satisfies the goal




```
(:goal (and (on s1)
            (on s2)
            (on s3))
)
```

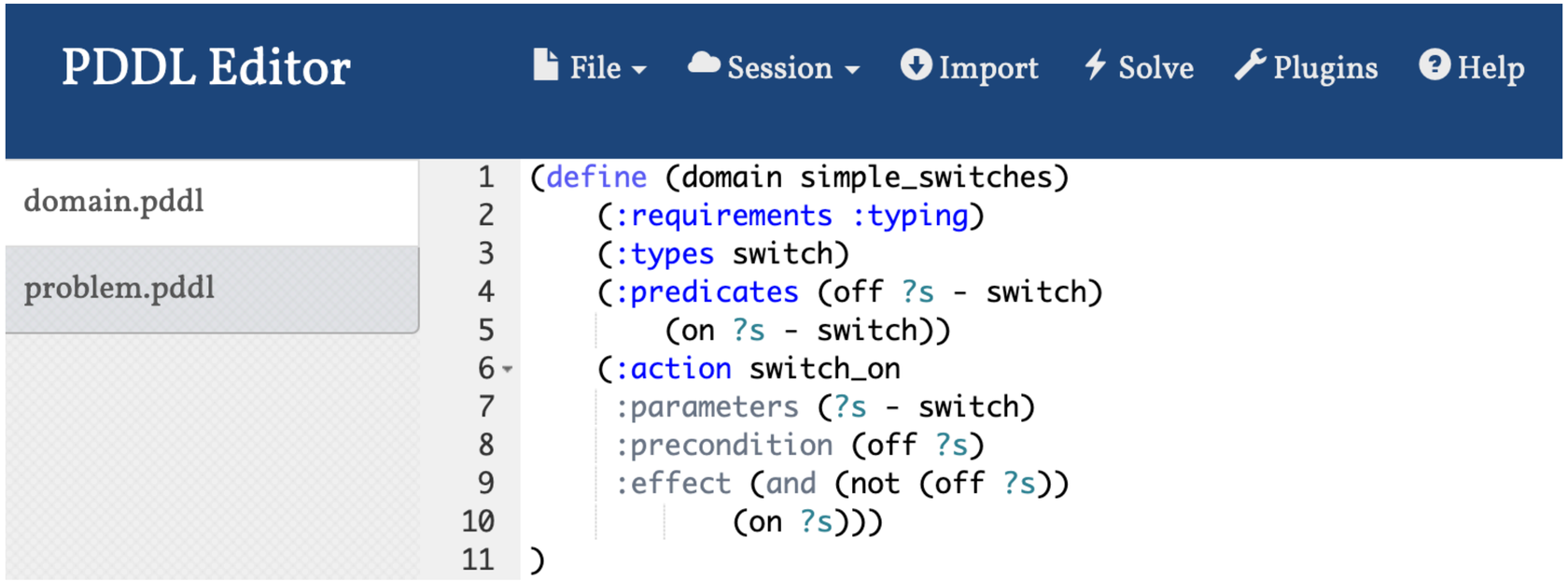
PLAN:

```
(switch_on s1)
(switch_on s2)
```

Classic PDDL - Example

A plan for a classical planning problem is a sequence of actions that are applicable from the initial state and lead to a a state that satisfies the goal

Online PDDL Editor: to write and solve PDDL planning problems:

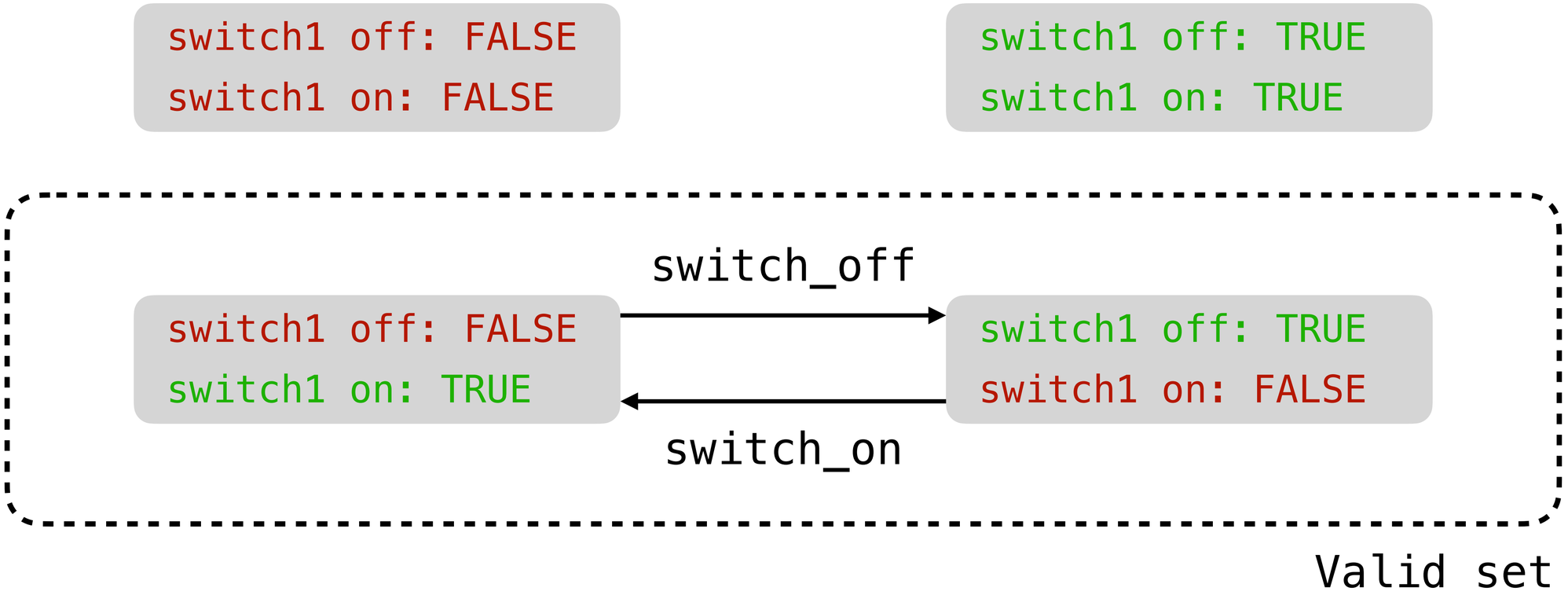


Did you create the switch_off action?

```
(:action switch_off
    :parameters (?s - switch)
    :precondition (on ?s)
    :effect (and (not (on ?s))
                 (off ?s))))
```

http://editor.planning.domains/#edit_session=wQgyhJKiR0wyquH

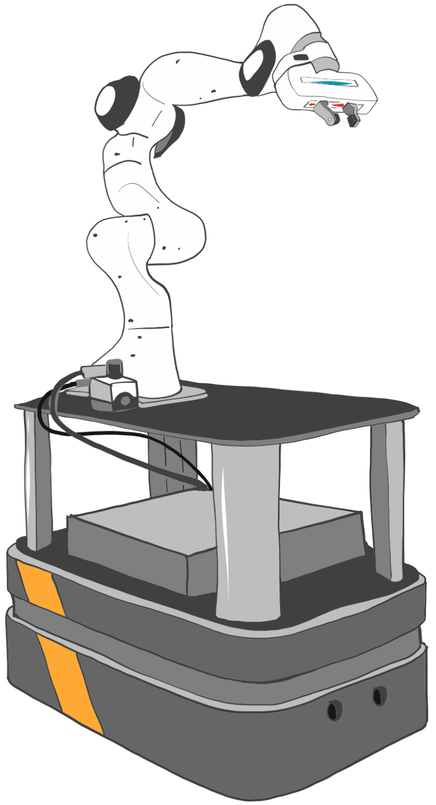
Add and delete effects



Temporal Planning - PDDL 2.1

- Up until now we considered classical planning with instantaneous actions
- For things to happen at the same time we need to model duration and concurrency of actions and events
- Complex inter-dependancies which determine what is physically possible
  - Example:

For a mobile manipulator navigating a store it might be possible to scan the environment while moving, but it might be impossible to attempt to pick something if not still



PDDL 2.1 - Tuple

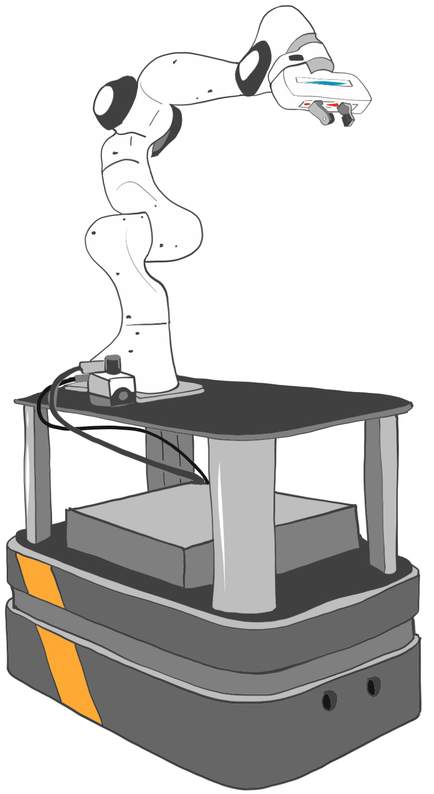
- PDDL 2.1 extends PDDL to more realistic scenarios where actions have durations and adds numeric variables and optimisation
- The temporal planning problem is a tuple: $< F, V, A, I, G >$
- where:
  - F is a set of boolean propositions
  - V is a set of real numeric expressions
  - A is a set of deterministic actions which can be durative
  - I is the initial state and G is the goal

PDDL 2.1 - State and solution

- The state is now a tuple $S = (t,\ s_l,\ s_n)$:
  - $t \in \mathbb{R}$
  - Logical state $s_l \in F$
  - Numerical state $s_n : V \to \mathbb{R}_\perp$ (real numbers + *undefined* if not specified)
- Solution to a temporal plan:
  - No longer a simple sequence of actions
  - Timed schedule of $(t_i,\ a_i)$

Durative actions

- A durative action consists now of:
  - Action name
  - Parameters
  - Duration
  - Conditions: `at start`, `over all`, `at end`
  - Effects: `at start`, `at end`



PDDL 2.1 - Syntax

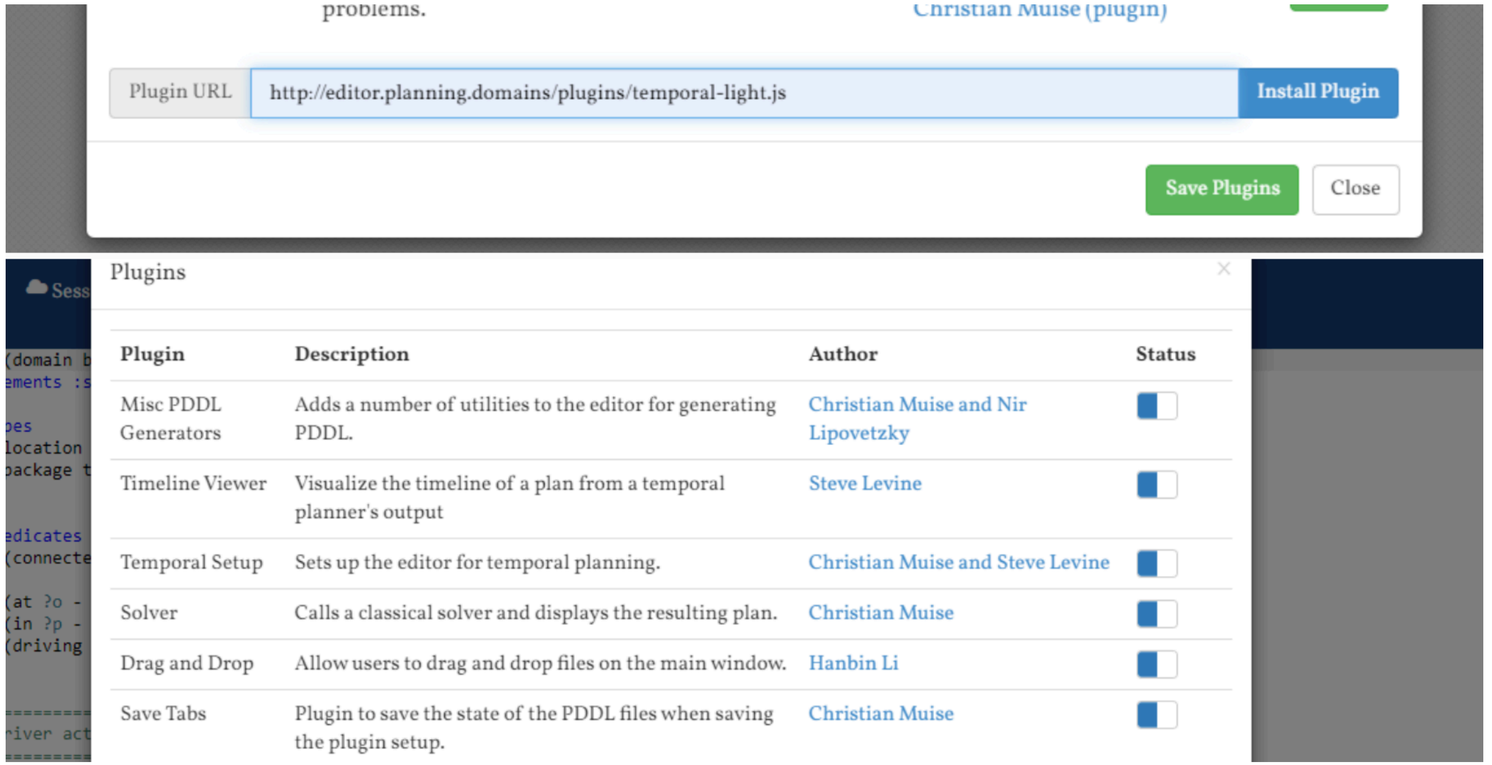
Durative Action

```
(:durative-action zoom_meeting
   :parameters (?s - student
                ?t - teacher)
   :duration (<= ?duration 120)
   :condition (and
     (at start (awake ?s))
     (over all (awake ?t)))
   :effect (and
     (at end (not (awake ?s)))))
```

- What is different from classical PDDL?
  - `:durative-action`
  - `:duration` - expressed as comparison (< , > , = , > = , < =)
  - `:condition` - not preconditions anymore
  - `:effects` - they happen either at start or at end

PDDL 2.1 - Online editor

PDDL Editor supports temporal planning with the right plugins:



http://editor.planning.domains/#read_session=EWjbgnhuUd

Exercise (30 min)

Imagine you are in charge of the stage lighting instruments to illuminate important live performance at international concerts. Your task for a particular venue in Moscow is to make sure that all 5 available stage lights are ON whenever the artist enters the stage. A colleague of yours provided you with the following PDDL files to solve the problem for a generic venue with 5 lights:

Unfortunately, the electrical system in Moscow was not installed properly and you cannot independently switch ON and OFF single lights as you please. In particular, a light can be switched ON if and only if there is already another adjacent light ON. Your task is to modify the given domain.pddl and problem.pddl templates and generate a plan to turn ON all the lights respecting the constraints. The initial state is the following:

- Initial state: {off, on, off, off, on}

ROSPlan

A ROS module to integrate planning into robotics for plan execution.

Planning in Robotics

Planning can be pretty abstract

- How can a plan be used to perform a task in the real world?

So far

- Platform
- Problem and Domain
- Generate a solution

Missing

- Update and maintain the world model
- Process and dispatch the plan



Planning in Robotics

Using task planning in robotics might be tedious

- A lot of different things need to be integrated:
  - State estimation - build and update a symbolic representation of the world
  - Domain and problem definition for the task
  - Generate the plan
  - Execute the plan according to runtime data

ROSPlan facilitates this integration

ROSPlan - Overview

PS: planning system
KB: knowledge base

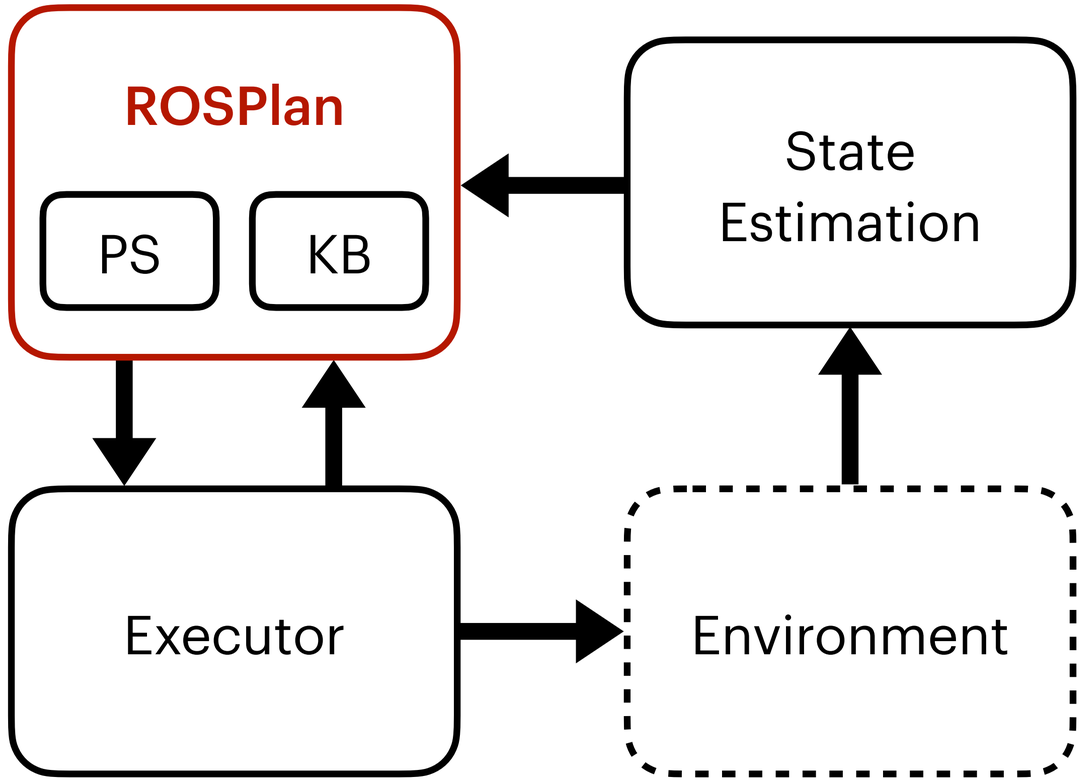
ROSPlan integrates standard tools:

- Planning with PDDL 2.1
- ROS (Robot Operating System)

ROSPlan takes care of:

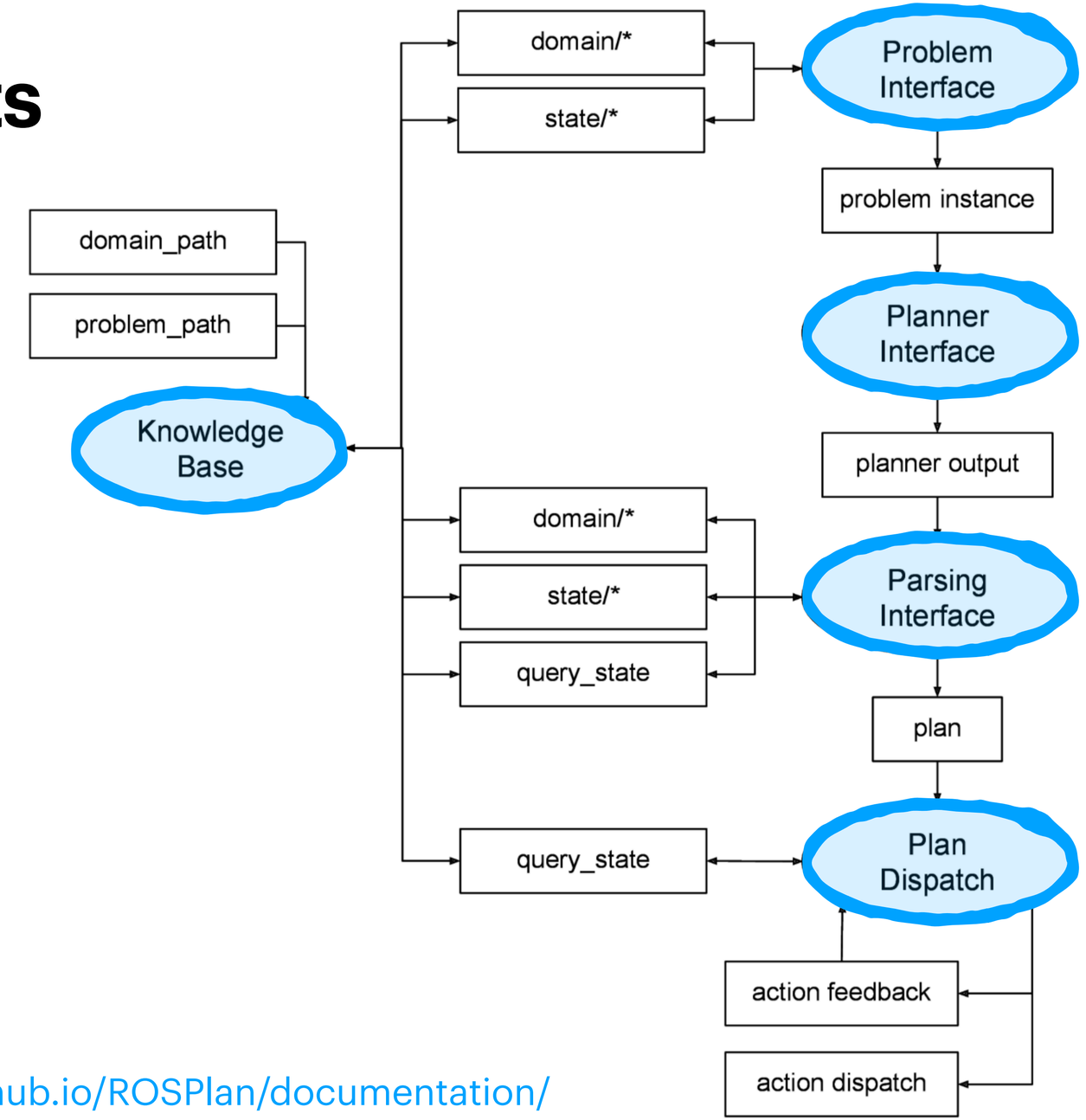
- Build the initial state from the PDDL problem
- Generate and validate the plan
- Dispatch the plan

ROSPlan is domain/platform generic



ROSPlan - Components

- The Knowledge Base
- The Problem Interface
- The Planner Interface
- The Parsing Interface
- The Plan Dispatch

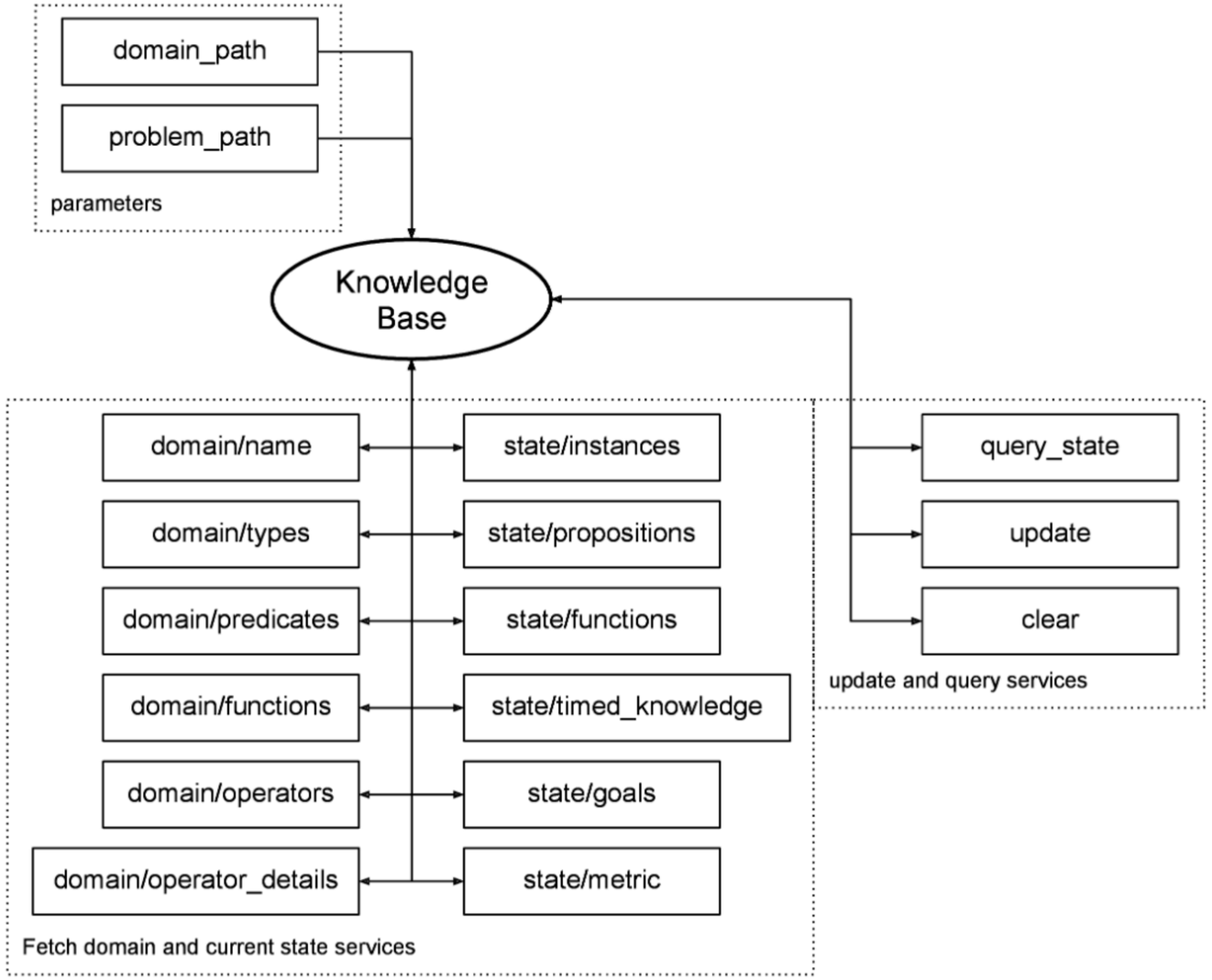


Resources:
https://kcl-planning.github.io/ROSPlan/documentation/

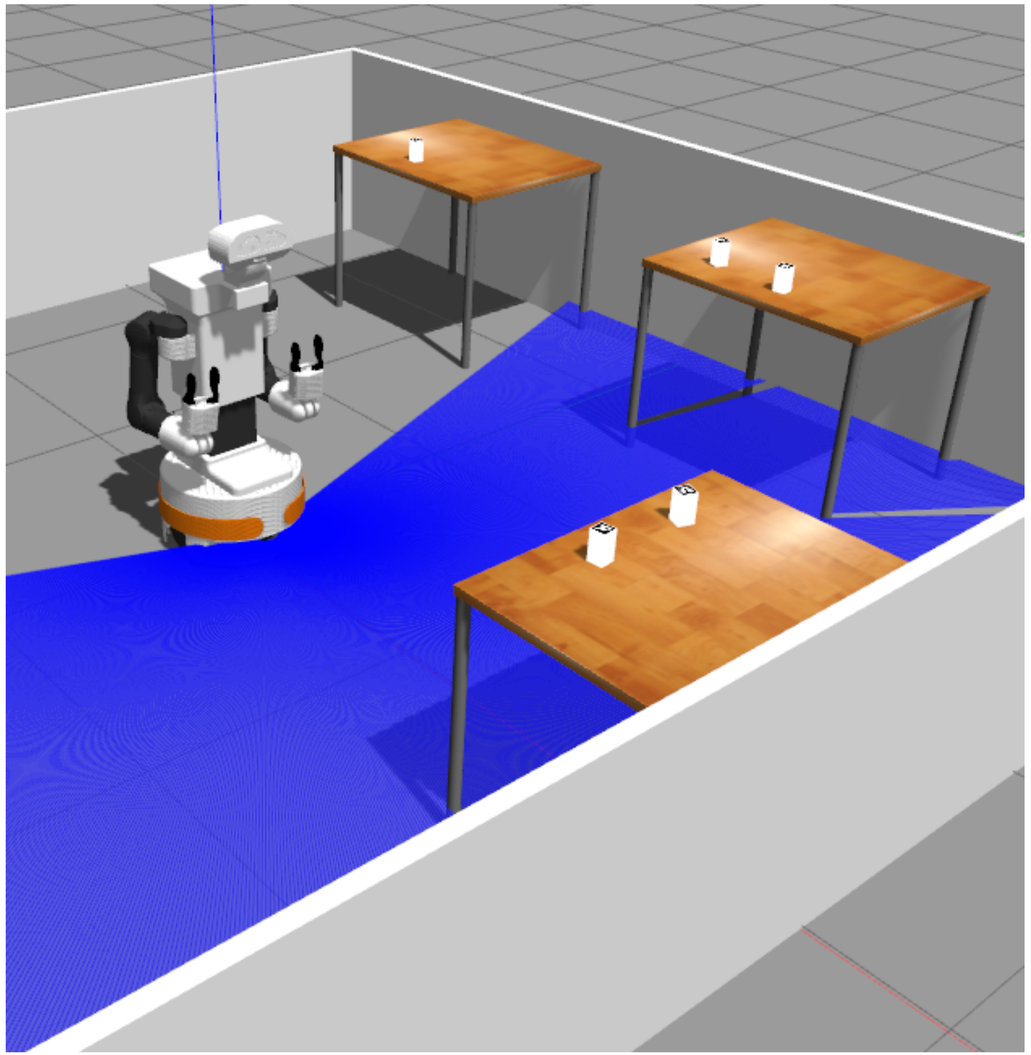
ROSPlan - Knowledge Base

The Knowledge Base:

- Loads a PDDL domain and problem from `.pddl` files
- Stores the state as a PDDL instance
- Is updated by ROS messages
- Can be queried



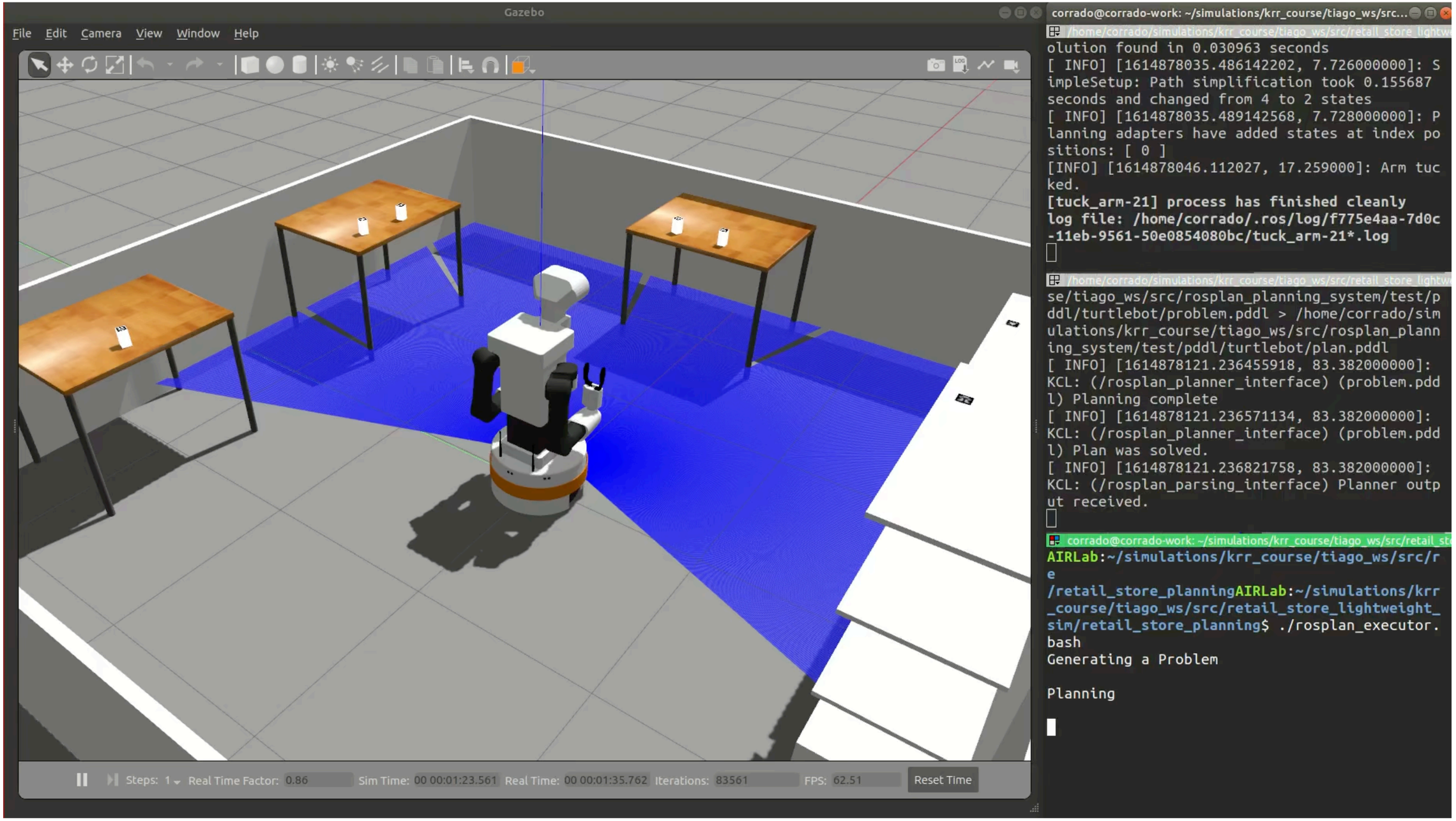
ROSPlan - Example



Pick *some* object from *somewhere*

Domain file
`domain.pddl`

Problem file
`problem.pddl`


Gazebo
File Edit Camera View Window Help
Steps: 1 Real Time Factor: 0.86 Sim Time: 00 00:01:23.561 Real Time: 00 00:01:35.762 Iterations: 83561 FPS: 62.51 Reset Time
corrado@corrado-work: ~/simulations/krr_course/tiago_ws/src...
olution found in 0.030963 seconds
[INFO] [1614878035.486142202, 7.726000000]: SimpleSetup: Path simplification took 0.155687 seconds and changed from 4 to 2 states
[INFO] [1614878035.489142568, 7.728000000]: Planning adapters have added states at index positions: [0]
[INFO] [1614878046.112027, 17.259000]: Arm tucked.
[tuck_arm-21] process has finished cleanly
log file: /home/corrado/.ros/log/f775e4aa-7d0c-11eb-9561-50e0854080bc/tuck_arm-21*.log
se/tiago_ws/src/rosplan_planning_system/test/pddl/turtlebot/problem.pddl > /home/corrado/simulations/krr_course/tiago_ws/src/rosplan_planning_system/test/pddl/turtlebot/plan.pddl
[INFO] [1614878121.236455918, 83.382000000]: KCL: (/rosplan_planner_interface) (problem.pddl) Planning complete
[INFO] [1614878121.236571134, 83.382000000]: KCL: (/rosplan_planner_interface) (problem.pddl) Plan was solved.
[INFO] [1614878121.236821758, 83.382000000]: KCL: (/rosplan_parsing_interface) Planner output received.
corrado@corrado-work: ~/simulations/krr_course/tiago_ws/src/retail_st
AIRLab:~/simulations/krr_course/tiago_ws/src/re
/retail_store_planningAIRLab:~/simulations/krr_course/tiago_ws/src/retail_store_lightweight_sim/retail_store_planning$ ./rosplan_executor.bash
Generating a Problem
Planning

DEMO

Solution for Robotic gripper manipulating boxes: http://editor.planning.domains/#edit_session=6GXQPcJRtBZVWJN

Visualisation: https://blocks-dot-ai-planning.appspot.com/